

车型图片筛选工具 - 操作手册

📋 工具选择指南

需求	使用工具	说明
快速检测单张/多张图片是否通过过滤	<code>car_image_checker.py</code>	简单命令行工具，直接传路径即得结果
批量筛选+下载+替换到品牌目录	<code>car_image_filter_pipeline.py</code>	完整流水线，支持3种输入方式

⚡ 快速开始

```
# 快速检查图片（最简单）
python scripts/car_image_checker.py photo.jpg

# 批量处理文件夹并生成报告
python scripts/car_image_checker.py D:\images --report report.html

# 完整流水线：筛选+替换（需指定品牌型号）
python scripts/car_image_filter_pipeline.py --step all \
  --source-dir "D:\images" --brand porsche --model 911
```

Part 1：快速检测工具（car_image_checker.py）

适用于：快速检查图片是否符合纯白背景 + 完整车身要求

一、快速上手

```
# 单张图片
python scripts/car_image_checker.py photo.jpg

# 多个图片
python scripts/car_image_checker.py 1.jpg 2.jpg 3.jpg

# 整个文件夹
python scripts/car_image_checker.py D:\my_images

# 递归扫描子文件夹
python scripts/car_image_checker.py D:\my_images --recursive
```

二、常用场景

场景1：批量检查并保存通过的图片

```
python scripts/car_image_checker.py D:\my_images --output D:\passed_images
```

自动将通过过滤的图片复制到 `D:\passed_images` 目录。

场景2：生成HTML可视化报告

```
python scripts/car_image_checker.py D:\my_images --report report.html
```

生成可离线查看的HTML报告，包含图片预览和详细检测指标。

场景3：显示详细检测指标

```
python scripts/car_image_checker.py D:\my_images --verbose
```

输出每张图片的白色占比、车宽/车高占比等详细数据。

三、命令参数

参数	说明	默认值
<code>paths</code>	图片文件或文件夹路径（支持多个）	必填
<code>--recursive</code> / <code>-r</code>	递归扫描子文件夹	False
<code>--output</code> / <code>-o</code>	将通过的图片复制到此目录	无
<code>--report</code>	生成HTML可视化报告	无
<code>--verbose</code> / <code>-v</code>	显示详细检测指标	False

四、输出示例

 发现 10 张图片，开始检测...

[1/10]  01_white_complete.png | 白75.3% 车宽83.4%

[2/10]  02_white_narrow.png | 白94.8% 车宽16.8%

...

=====

检测结果: 1 通过 / 9 不通过 / 共 10 张

=====

 通过 1 张:

01_white_complete.png1200x80075.3%83.4%36.4%

 不通过 9 张:

02_white_narrow.png→ 车身(车辆宽度占比异常(17%))

06_gray_complete.png→ 背景(4角非白色, 白色占比低)

...

五、退出码

退出码	说明
0	至少有一张图片通过
1	没有图片通过, 或未找到任何图片

Part 2: 完整流水线工具

(car_image_filter_pipeline.py)

适用于：从下载/导入到替换的完整流程，支持批量处理多车型

一、工具简介

本工具用于从大量候选图片中，自动筛选出符合以下条件的车型正侧视图：

- 纯白背景** — 4角颜色一致、亮度足够、白色像素占比高
- 完整车身** — 车辆宽度占比 50-98%、左右上下都有留白
- 车头向左** — 原图或水平翻转版二选一

最终将筛选出的图片替换到品牌目录 `public/brands/`。

全自动四重过滤流程

原始图片 → [步骤1：纯白背景过滤] → [步骤2：完整车身检测] → [步骤3：车头方向自动判定] → 替换
(代码自动) (代码自动) (算法自动，四层特征融合)

车头方向自动判定算法（无需人工确认）：

特征	权重	原理
底部轮廓车轮检测	3	车轮触地=车身最低点（颜色无关）
自适应亮度车轮检测	1	车轮=底部最暗区（相对中值30%）
上轮廓非对称性	1	引擎盖低于行李箱/C柱
亮度梯度	1	车头更亮（反射/格栅），亮度重心偏向车头

可靠性检查：底部平坦、车轮过宽或间距过小时自动降级到亮度梯度。测试准确率100%（4款车×8张图）。

目录结构

```
public/
├── _bing_v2_candidates/           # Bing下载的原始候选图（或本地导入的图）
│   └── bentley__bentayga/
├── _white_bg_candidates/        # 纯白背景过滤通过的图
│   └── bentley__bentayga/
├── _complete_car_candidates/    # 完整车身检测通过的图（含原图orig_和翻转版flip_）
│   └── bentley__bentayga/
│       ├── orig_6bd7cf90.jpg
│       └── flip_6bd7cf90.jpg
└── brands/                     # 最终替换的品牌图
    └── bentley/
        └── bentayga.jpg
```

二、环境准备

依赖安装

```
pip install Pillow numpy
```

启动开发服务器（预览页面需要）

```
npm run dev
```

预览页面访问地址前缀：<http://localhost:5173/>

三、三种运行方式

方式1：本地图片文件夹（最常用）

适用于：你已经有某车型的图片文件夹，想直接筛选替换

一键全流程

```
python scripts/car_image_filter_pipeline.py --step all \
    --source-dir "D:\下载\保时捷911图片" \
    --brand porsche --model 911 --name "Porsche 911"
```

参数说明:

参数	说明	示例
--source-dir	本地图片文件夹路径	D:\下载\保时捷911图片
--brand	品牌名 (英文, 用于目录名)	porsche
--model	型号名 (英文, 用于文件名)	911
--name	显示名 (可选, 默认用 brand model)	Porsche 911

支持的图片格式: .jpg .jpeg .png .webp .bmp

分步执行

```
# 步骤1: 导入本地图片
python scripts/car_image_filter_pipeline.py --step download \
    --source-dir "D:\下载\保时捷911图片" \
    --brand porsche --model 911

# 步骤2: 纯白背景过滤
python scripts/car_image_filter_pipeline.py --step filter \
    --source-dir "D:\下载\保时捷911图片" \
    --brand porsche --model 911

# 步骤3: 完整车身检测 + 生成翻转版
python scripts/car_image_filter_pipeline.py --step complete \
    --source-dir "D:\下载\保时捷911图片" \
    --brand porsche --model 911

# 步骤4: 生成预览页
python scripts/car_image_filter_pipeline.py --step preview \
    --source-dir "D:\下载\保时捷911图片" \
    --brand porsche --model 911

# 步骤5: 替换到品牌目录（自动选白色占比最高的）
python scripts/car_image_filter_pipeline.py --step apply --apply-best \
    --source-dir "D:\下载\保时捷911图片" \
    --brand porsche --model 911
```

方式2: 批量自动识别（多车型）

适用于：一次处理多个车型，每个车型一个子文件夹

文件夹结构要求

```
D:\car_images\  
├─ porsche_911\          ← Porsche 911 的图片  
│   ├── img1.jpg  
│   ├── img2.png  
│   └── ...  
├─ ferrari_f8\          ← Ferrari F8 的图片  
│   ├── img1.jpg  
│   └── ...  
└─ lamborghini_aventador\ ← Lamborghini Aventador 的图片  
    ├── img1.jpg  
    └── ...
```

子文件夹命名规则： `品牌_型号`（用下划线分隔，或用双下划线 `品牌__型号`）

一键全流程

```
python scripts/car_image_filter_pipeline.py --step all \  
    --source-dir "D:\car_images" \  
    --auto-detect
```

脚本会自动遍历 `D:\car_images` 下的所有子文件夹，将每个子文件夹作为一个车型处理。

品牌和型号识别规则： 子文件夹名按第一个下划线分割，前半部分为品牌，后半部分为型号。例如 `porsche_911` → 品牌 `porsche`，型号 `911`。

分步执行

```
# 导入 + 过滤 + 检测 + 预览  
python scripts/car_image_filter_pipeline.py --step all \  
    --source-dir "D:\car_images" --auto-detect  
  
# 算法自动判定方向后，批量替换（无需人工确认）  
python scripts/car_image_filter_pipeline.py --step apply --apply-best \  
    --source-dir "D:\car_images" --auto-detect
```

方式3：从Bing下载（需配置CARS列表）

适用于：没有现成图片，需要从搜索引擎下载候选图

步骤1：添加车型配置

打开 `scripts/car_image_filter_pipeline.py` , 在 `CARS` 列表中添加车型:

```
CARS = [  
    # ... 已有车型 ...  
    {  
        "brand": "porsche",  
        "model": "911",  
        "name": "Porsche 911",  
        "cn_name": "保时捷911",  
        "queries": [  
            "{name} side profile official photo",  
            "{name} side view white background",  
            "{cn_name} 侧面 官方图片 纯色背景",  
        ],  
    },  
]
```

字段说明:

字段	说明
<code>brand</code>	品牌名 (英文, 用于目录名)
<code>model</code>	型号名 (英文, 用于文件名)
<code>name</code>	显示名 (英文)
<code>cn_name</code>	中文名 (用于中文搜索关键词)
<code>queries</code>	搜索关键词列表, <code>{name}</code> 替换为英文名, <code>{cn_name}</code> 替换为中文名

步骤2: 运行

```
# 处理CARS列表中的所有车型  
python scripts/car_image_filter_pipeline.py --step all  
  
# 只处理指定车型  
python scripts/car_image_filter_pipeline.py --step all --car porsche/911  
  
# 强制重新下载 (忽略已下载的候选图)  
python scripts/car_image_filter_pipeline.py --step all --car porsche/911 --force
```

四、完整操作流程（3步走）

无论使用哪种输入方式，核心流程都是3步：

第1步：筛选（代码自动）

```
python scripts/car_image_filter_pipeline.py --step all \  
    --source-dir "D:\my_images" --brand porsche --model 911
```

自动执行：

1. 导入图片（本地文件夹 或 Bing下载）
2. 纯白背景过滤（4角检测 + 亮度差异 + 白色占比）
3. 完整车身检测（宽度占比 + 留白检查）
4. 生成水平翻转版（供方向选择）
5. 生成预览页面

输出示例：

```
--- Porsche 911 ---  
  
[Porsche 911] 本地导入完成：15张（源：D:\my_images）  
  
[Porsche 911] 纯白背景过滤：6/15张通过  
  
[Porsche 911] 完整车身检测：3/6张通过  
  
  
预览页：http://localhost:5173/\_preview\_pipeline.html
```

第2步：车头方向自动判定（算法，无需人工）

主流程内置四层特征融合算法（底部轮廓车轮检测/自适应亮度/上轮廓非对称性/亮度梯度），在 `--step all` 或 `--step apply` 时自动判定方向，并选择原图或翻转版应用。

预览页面：http://localhost:5173/_preview_pipeline.html（仅供事后复核，不参与决策）

第3步：替换（代码自动）

```
# 自动应用白色占比最高 + 算法自动判定方向的候选  
  
python scripts/car_image_filter_pipeline.py --step apply --apply-best \  
    --source-dir "D:\my_images" --brand porsche --model 911
```

v3流程已完全自动化：算法会根据四层特征融合判定车头方向，自动选择原图或翻转版应用，无需手动指定翻转标记。

五、过滤参数调优

如果筛选结果不理想，可以调整过滤参数。所有参数集中在脚本顶部的两个配置块中。

纯白背景过滤参数

```
WHITE_BG_CONFIG = {  
    "corner_std_threshold": 12,      # 4角颜色标准差阈值（越小越严格）  
    "corner_brightness_diff": 15,    # 4角最大亮度差阈值（防止渐变背景）  
    "min_brightness": 235,           # 最小亮度（白色背景）  
    "min_white_ratio": 0.55,         # 最小白色像素占比（55%）  
    "min_width": 800,                # 最小宽度（像素）  
    "asp_min": 1.3,                  # 宽高比下限  
    "asp_max": 2.5,                  # 宽高比上限  
    "corner_size": 15,               # 角落采样区域大小  
    "white_threshold": 230,          # 白色判定阈值（RGB各分量）  
    "use_edge_detection": False,     # 是否启用边缘条带检测  
}
```

常见调优场景：

问题	调整方法
白色背景被误杀（太严格）	降低 <code>min_white_ratio</code> 到 0.45，或提高 <code>corner_std_threshold</code> 到 20
非白色背景被放过（太宽松）	提高 <code>min_brightness</code> 到 240，或降低 <code>corner_std_threshold</code> 到 8
渐变背景被放过	降低 <code>corner_brightness_diff</code> 到 10
小图被放过	提高 <code>min_width</code> 到 1000

完整车身检测参数

```
COMPLETE_CAR_CONFIG = {  
    "min_width_ratio": 0.50,         # 车辆宽度最小占比（50%）  
    "max_width_ratio": 0.98,         # 车辆宽度最大占比（98%）  
    "min_height_ratio": 0.30,        # 车辆高度最小占比（30%）  
    "max_height_ratio": 0.90,        # 车辆高度最大占比（90%）  
    "min_margin": 5,                 # 最小左右留白（像素）  
    "min_top_margin": 3,              # 最小上留白（像素）  
    "min_bottom_margin": 3,           # 最小下留白（像素）  
    "white_threshold": 230,          # 白色判定阈值  
}
```

常见调优场景：

问题	调整方法
局部特写被放过	提高 <code>min_height_ratio</code> 到 0.40
车辆太小的图被放过	提高 <code>min_width_ratio</code> 到 0.60
边缘略裁切的图被误杀	降低 <code>min_margin</code> 到 2

六、命令参数速查表

参数	说明	默认值
<code>--step</code>	执行步骤：all/download/filter/complete/apply/preview	all
<code>--source-dir</code>	本地图片文件夹路径（跳过Bing下载）	无
<code>--brand</code>	品牌名（命令行指定，无需改CARS配置）	无
<code>--model</code>	型号名	无
<code>--name</code>	显示名	brand + model
<code>--car</code>	筛选CARS列表中已配置的车型（格式：brand/model）	无
<code>--auto-detect</code>	从source-dir子文件夹名自动识别车型	False
<code>--force</code>	强制重新下载	False
<code>--apply-best</code>	自动应用白色占比最高的候选	False

七、常见问题

Q1: 运行报错 `ModuleNotFoundError: No module named 'PIL'`

```
pip install Pillow numpy
```

Q2: 预览页面打不开

确保开发服务器正在运行：

```
npm run dev
```

Q3: 纯白背景过滤通过0张

可能原因：

1. 图片背景不是纯白（检查4角是否为白色）
2. 图片尺寸太小（检查 `min_width` 参数）
3. 宽高比不符合（检查 `asp_min/asp_max` 参数）

调试方法： 运行测试脚本查看每张图的过滤指标

```
python scripts/test_filter_logic.py
```

Q4: 完整车身检测通过0张

可能原因：

1. 车辆占图片比例太小（检查 `min_width_ratio`）
2. 车辆边缘贴边（检查 `min_margin`）
3. 图片是局部特写而非整车（检查 `min_height_ratio`）

Q5: 替换后图片方向不对（车头向右）

v3算法已通过19款车×38用例100%准确率回归测试，正常情况下不会出现此问题。

若仍遇到（如上传了非标准正侧视图），可手动覆盖：

```
step_apply(car, "文件名.jpg", flip=True)
```

Q6: Bing下载0张图

可能原因：

1. 网络问题（检查能否访问 `cn.bing.com`）
 2. 搜索关键词不合适（尝试更通用的关键词）
 3. Bing页面结构变化（检查 `bing_search_images` 的正则匹配）
-

八、文件清单

文件	说明
scripts/car_image_checker.py	快速检测工具（单图/多图/文件夹）
scripts/car_image_filter_pipeline.py	完整流水线工具（筛选+替换）
scripts/test_filter_logic.py	过滤逻辑测试脚本（10张测试图）
scripts/_test_filter_images.zip	测试图片打包
public/_test_filter_report.html	测试报告（可视化）
public/_preview_pipeline.html	筛选结果预览页面